Deep Learning-Based Side-Channel Attack on AES Using Power Traces

Aarush Mohan Mathur
Dept. of Computer Science
Shiv Nadar University
2410110391

Sarthak Sethi
Dept. of Computer Science
Shiv Nadar University
2410110472

Adya Singhal
Dept. of Computer Science
Shiv Nadar University
2410110027

Shaurya Ajit Singh
Dept. of Computer Science
Shiv Nadar University
2410110318

***Abstract*—Microarchitectural side-channel attacks exploit shared hardware resources, such as CPU caches and branch predictors, to leak sensitive information across process boundaries. Techniques like Prime+Probe and Flush+Reload induce subtle variations in cache access patterns, making them difficult to detect using conventional software-level defenses. This work proposes a runtime detection framework for identifying cache-based and memory-based side-channel activities using Hardware Performance Counters (HPCs) available in modern processors. The framework collects low-level microarchitectural metrics, including cache misses, cache references, branch mispredictions, context switches, and memory access events, via performance monitoring interfaces such as perf and PMUs. These features are analyzed to distinguish benign workloads from adversarial behavior using statistical profiling and machine learning-based classification models. The system is evaluated in terms of detection accuracy, false positive rates, and generalization across diverse workloads and system conditions. By leveraging hardware-level observability, the proposed approach provides a lightweight and deployable mechanism for detecting side-channel attacks in real time. This work contributes toward strengthening operating system-level defenses against microarchitectural leakage and demonstrates the feasibility of HPC-driven anomaly detection for securing multi-tenant computing environments.**

***Index Terms*—Cryptographic security, implementation leakage, side-channel analysis, deep learning, AES, microarchitectural attacks, cache side channels, Prime+Probe, Flush+Reload, cross-process leakage, Hardware Performance Counters, anomaly detection, machine learning, runtime security, system security**

I. Introduction

Modern cryptographic systems such as the Advanced Encryption Standard (AES) are widely regarded as mathematically secure against classical cryptanalysis. However, practical implementations often leak sensitive information through unintended physical and microarchitectural channels. This leakage enables side-channel attacks (SCAs), which exploit measurable characteristics such as execution time, power consumption, electromagnetic emissions, and memory access patterns to infer secret data without directly breaking the underlying cryptographic primitive [4].

The growing adoption of machine learning techniques further amplifies this threat. Learning-based models can automatically capture complex, non-linear relationships in side-channel data, significantly improving the efficiency and success rate of attacks against otherwise secure implementations [2]. In particular, cache-based attacks such as *Prime+Probe* and *Flush+Reload* [7] manipulate and monitor shared cache states to extract sensitive information across process boundaries. These techniques are especially effective in multi-tenant environments, where adversaries can co-locate malicious processes with victim workloads and observe microarchitectural interactions to reconstruct secret data [3]–[5]. Notably, such attacks do not require physical proximity or specialized measurement equipment, making them highly practical and scalable in real-world systems.

Despite extensive research on attack methodologies, detecting microarchitectural side-channel activity in operational systems remains a significant challenge. Unlike conventional attacks, SCAs rely on subtle variations in hardware behavior that are often indistinguishable from normal program execution, resulting in a low signal-to-noise ratio [1], [3], [6]. Traditional security mechanisms primarily focus on enforcing access control or detecting explicit software-level anomalies and therefore fail to capture leakage arising from legitimate execution patterns. Furthermore, benign applications can exhibit cache and memory access behaviors similar to adversarial workloads, leading to high false positive rates in naive detection approaches [1].

Hardware Performance Counters (HPCs) provide a promising mechanism for observing low-level processor activity. These counters offer fine-grained measurements of events such as cache misses, memory accesses, and branch mispredictions. Prior research suggests that side-channel attacks induce characteristic patterns in these microarchitectural metrics [1]–[7]. By analyzing these patterns, it becomes possible to identify deviations indicative of cache-based or memory-based attacks. In particular, combining HPC-derived features with statistical analysis and machine learning techniques enables the development of classifiers capable of detecting anomalous execution behavior at runtime.

These observations highlight a critical limitation: cryptographic strength at the algorithmic level does not guarantee security at the implementation level. Therefore, securing modern systems requires addressing not only theoretical vulnerabilities but also leakage arising from real-world execution environments.

II. Literature Review

1) **AIMLSYS (IITB)** [1]

Summary: Provides a foundational overview of applying machine learning to side-channel attacks, emphasizing profiling attacks and the transition from classical statistical techniques to ML-based approaches. It highlights how ML models can learn leakage functions directly from traces, improving automation and scalability.

2) **arXiv:2404.11284** [2]

Summary: Surveys modern deep learning-based SCA techniques, focusing on profiling attacks, neural architectures, and emerging methods such as synthetic trace generation. It shows how DL reduces reliance on handcrafted leakage models and improves robustness to noise.

3) **PROOFS Workshop Paper** [3]

Summary: Discusses formal and practical challenges in side-channel security evaluation, including leakage modeling and validation. Emphasizes the difficulty of proving resistance against ML-based attacks.

4) **IACR 2018/053 (ASCAD paper)** [4]

Summary: Introduces the ASCAD dataset, a standardized benchmark for deep learning-based SCA. Demonstrates how CNNs and MLPs can recover AES keys from power traces and establishes profiling attacks as a dominant paradigm.

5) **arXiv:2603.10726** [5]

Summary: Presents recent advancements in ML-driven SCA, focusing on improved architectures and attack strategies. Highlights scalability challenges and efficiency trade-offs in modern deep learning attacks.

6) **arXiv:2603.18647** [6]

Summary: Explores cutting-edge SCA techniques, including hybrid and adaptive learning-based attacks. Emphasizes generalization issues and robustness across different devices and datasets.

7) **UKCI 2017 Paper** [7]

Summary: One of the early works applying machine learning to SCA, demonstrating feasibility of classification-based leakage exploitation. Establishes groundwork for later deep learning approaches.

8) **arXiv:2103.14244** [8]

Summary: Analyzes deep learning architectures in SCA, focusing on CNN effectiveness and preprocessing techniques. Shows that proper data representation significantly impacts attack success.

9) **IACR 2025/014** [9]

Summary: Recent work analyzing advanced SCA techniques and modern attack strategies. Focuses on improving generalization and reducing data requirements in profiling attacks.

10) **arXiv:2312.11094** [10]

Summary: Explores modern ML-based SCA improvements, including architectural innovations and optimization techniques. Highlights efficiency vs accuracy tradeoffs.

11) **arXiv:2309.13170** [11]

Summary: Investigates efficient deep learning architectures for AES SCA using ASCAD. Demonstrates that optimized models can reduce computational cost while maintaining attack performance.

12) **HOST 2023 Paper** [12]

Summary: Focuses on hardware-level side-channel leakage and detection using performance metrics. Highlights the role of hardware behavior in enabling side-channel attacks.

13) **arXiv:1605.00681** [13]

Summary: Early deep learning SCA work showing that neural networks can outperform classical attacks without explicit leakage modeling, marking a shift toward data-driven approaches.

14) **arXiv:1611.03748** [14]

Summary: Demonstrates the effectiveness of deep neural networks in side-channel attacks, particularly in handling noisy and misaligned traces.

15) **ScienceDirect (2023 Review)** [15]

Summary: Provides a comprehensive review of AES attacks, including side-channel, fault injection, and ML-based methods. Highlights the evolution from classical CPA/DPA to AI-driven attacks and emphasizes the growing importance of deep learning in modern cryptanalysis.

16) **AISY Framework** [16]

Summary: Provides an experimental framework for conducting SCA using machine learning. Enables reproducible evaluation of models using metrics such as GE and SR.

17) **ASCAD Dataset** [17]

Summary: Standard benchmark dataset for profiling SCA, widely used for evaluating ML/DL models. Provides labeled power traces for AES implementations, enabling reproducible research.

III. Background

A. Advanced Encryption Standard (AES) and Leakage

The Advanced Encryption Standard (AES) is a symmetric block cipher operating on 128-bit blocks with key sizes of 128, 192, or 256 bits. Its structure consists of multiple rounds involving *SubBytes*, *ShiftRows*, *MixColumns*, and *AddRoundKey* transformations. While AES is cryptographically secure at the algorithmic level, practical implementations exhibit data-

dependent behavior that can leak sensitive information through observable side channels [4], [8].

During execution, intermediate computations such as SubBytes and key XOR operations result in bit-level transitions. These transitions cause measurable variations in power consumption and electromagnetic emissions. By analyzing such side-channel information, attackers can correlate physical leakage with processed data values and ultimately recover secret keys. Leakage is commonly modeled using abstractions such as Hamming Weight (HW) and Hamming Distance (HD), which approximate the relationship between processed data and observed signals [9].

B. Side-Channel Attack Taxonomy

Side-channel attacks (SCAs) are broadly classified into profiling and non-profiling attacks. Profiling attacks assume access to a similar device and involve a training phase to learn leakage characteristics, often using statistical or machine learning techniques. In contrast, non-profiling attacks such as Differential Power Analysis (DPA) rely on statistical correlations between observed leakage and hypothetical key guesses without prior training [7].

The effectiveness of SCAs depends on multiple factors, including signal-to-noise ratio (SNR), guessing entropy (GE), success rate (SR), key ranking, and the presence of countermeasures. Strong leakage signals and accurate modeling significantly improve attack success rates.

C. Microarchitectural Side-Channel Attacks

Microarchitectural side-channel attacks exploit shared hardware resources such as caches, branch predictors, and translation lookaside buffers (TLBs) to infer sensitive information. These attacks arise due to contention and state changes in hardware structures and can be executed entirely in software without requiring specialized measurement equipment [8].

Cache-based attacks [10] are among the most prominent forms of such leakage. Techniques like *Prime+Probe* [3], [7] monitor cache usage patterns, while *Flush+Reload* exploits shared memory to observe fine-grained access behavior. These attacks are particularly effective in shared or multi-tenant environments, where adversaries can monitor victim processes indirectly [3].

D. Hardware Performance Counters (HPCs)

Hardware Performance Counters (HPCs) are specialized registers in modern processors that record low-level microarchitectural events such as cache accesses, cache misses, branch mispredictions, and instruction counts [1]. Although originally designed for performance analysis and optimization, HPCs provide valuable insights into program execution behavior.

By sampling HPCs over time, it is possible to construct a representation of program activity that reflects underlying microarchitectural interactions. These representations can be analyzed to detect deviations from expected behavior, making HPCs suitable for identifying anomalous activities such as side-channel attacks [8]. Furthermore, their accessibility through standard interfaces like `perf` enables deployment without requiring hardware modifications.

E. Power-Based Side-Channel Attacks

Power-based SCAs exploit variations in a device's power consumption during cryptographic execution to infer secret information such as encryption keys. The instantaneous power consumption of digital circuits depends on the operations performed, allowing attackers to recover intermediate values without breaking the underlying cryptographic algorithm [1], [4], [7], [13], [14].

Simple Power Analysis (SPA) involves direct interpretation of power traces, while Differential Power Analysis (DPA) applies statistical techniques over multiple traces to correlate power consumption with hypothetical intermediate values [4], [13]. These methods have proven highly effective against implementations of widely used algorithms such as AES [1], [9], [11].

F. Power Traces and Leakage Modeling

A power trace is a time-series representation of a device's power consumption during execution, typically obtained using high-resolution measurement equipment such as oscilloscopes [1], [4], [13]. Formally, a power trace can be represented as:

$$T = \{P(t_1), P(t_2), \ldots, P(t_n)\}$$

where each sample $P(t_i)$ corresponds to the power consumption at time instant t_i. Power traces contain both useful leakage information and noise [7], [8]. Multiple traces corresponding to different inputs are analyzed collectively to extract statistically significant patterns.

Leakage is commonly modeled as:

$$L = f(v) + n$$

where L represents observed leakage, v is an intermediate value dependent on secret data, and n denotes noise [4], [13]. Two widely used models are:

- **Hamming Weight (HW):** Number of bits set to 1 in a value, modeling static power consumption.
- **Hamming Distance (HD):** Number of bit transitions between two states, modeling dynamic switching activity.

These models approximate CMOS circuit behavior, where power consumption correlates with switching activity. They form the basis for attacks such as Correlation Power Analysis (CPA) [1], [7], [10].

G. Machine Learning in Side-Channel Analysis

Machine learning (ML) techniques enable direct learning of leakage patterns from data without requiring explicit leakage models. By treating side-channel analysis as a classification or regression problem, ML models can map power traces to secret-dependent variables or key hypotheses [2], [5], [8].

ML-based approaches are robust to noise, misalignment, and implementation-specific variations, reducing the number

of traces required for successful attacks [2], [8], [12]. Their ability to capture non-linear relationships makes them highly effective in practical scenarios.

H. Convolutional Neural Networks for SCA

Convolutional Neural Networks (CNNs) are particularly well-suited for side-channel analysis, as power traces can be interpreted as one-dimensional signals with localized temporal features. CNNs use convolutional filters to automatically extract relevant patterns such as peaks, transitions, and correlations associated with secret-dependent operations [2], [5], [12], [15].

Key advantages of CNNs include:

- Local feature extraction
- Translation invariance
- Noise robustness
- Automatic feature learning

These properties allow CNN-based methods to outperform traditional statistical techniques and other machine learning models, making them state-of-the-art for profiling-based side-channel attacks.

I. Leakage Modelling, Data Preprocessing, Normalization and Reshaping

Leakage modeling gives the mathematical relationship between secret-dependent intermediate values and the observable side-channel signal. In power-based SCA, the leakage is assumed to arise from data-dependent switching activity in CMOS circuits shown in instantaneous change in power usage. A general leakage model is defined as:

$$L(t) = f(v(t)) + n(t)$$

$L(t)$ = Observed Leakage ; $f(v(t))$: leakage functio of internal intermediate value ; $n(t)$ is the noise

For AES, intermediate value is:

$$v = Sbox(P \oplus K)$$

where P: Plaintext byte; K: 'Secret' key byte ; Sbox: AES Substitution operation

The Hamming Weight model:

$$HW(v) = \sum_{i=0}^{7} b_i$$

where b_i are bits of (v), thus converting static power into bits.

The Hamming Distance Model:

$$HD = HD(v_1, v_2) = \sum_{i=0}^{7} \left(b_i^{(1)} \oplus b_i^{(2)} \right)$$

which models dynamic power consumption based on bit transitions between successive states.

The classification target is thus:

$$y = HW(Sbox(P \oplus K)) \in \{0, 1, \ldots, 8\}$$

Hence $X \to Y$, i.e from a power problem to a classification problem.

Raw Side channel traces are noise due to measurement conditions, device characteristics, and execution jitter. Preprocessing is needed to enhance the signal-to-noise ratio (SNR) and enable stable learning. The dataset can be exhibited by:

$$X \in \mathbb{R}^{N \times T}$$

Where N: No of traces and T: Time sample per trace. Thus we achieve Noise reduction, Feature alignment and Scale normalization.

Additionally, the label vector:

$$Y \in \{0, \ldots, 255\}$$

is transformed using the leakage model into:

$$Y' \in \{0, \ldots, 8\}$$

Normalization is done to standardize data distribution of power traces for stability during training without which large magnitudes dominate learning. Used method was Z-score normalisation:

$$X_{\text{norm}}(i, j) = \frac{X(i, j) - \mu_j}{\sigma_j}$$

Where $\mu_j \to$ mean of time sample and $\sigma_j \to$ standard deviation.

Reshaping is required to adapt the one-dimensional power traces into a format compatible with convolutional neural networks (CNNs). While raw traces are represented as:

$$X \in \mathbb{R}^{N \times T}$$

CNNs expect input tensors with an explicit channel dimension:

$$X \in \mathbb{R}^{N \times T \times C}$$

; C is number of channels ($C = 1$ for power traces)

$$X_{\text{reshaped}} = \text{reshape}(X, (N, T, 1))$$

Applying Convolutional layers to apply filters:

$$y[n] = \sum_{k=0}^{K-1} w[k] \cdot x[n - k]$$

where $w[k]$ is the convolution kernel of size K.

IV. Methodology

A. Overview of Approach

This work adopts a profiling based SCA methodology to evaluate the effectiveness of Deep Learning models in extracting secret-dependent information from power consumption traces of an AES implementation. The approach follows a structured pipeline consisting of dataset acquisition, preprocessing, leakage modeling, neural network(nn) training and attack evaluation as shown in modern SCA frameworks [1], [4], [8], [12]. The ASCAD dataset [17] is used as the primary Source of Side channel Information.

The use of a leakage model is critical as it links observed data to secret dependent intermediate values. The Hamming Weight (HW) model is employed, where the leakage is assumed to be proportional to the number of bits set to one in the intermediate value. Specifically, the leakage is modeled as:

$$L = HW(Sbox(P \oplus K)) + n$$

P: Plaintext byte; K: 'Secret' key byte ; Sbox: AES Substitution operation; n: noise

The methodology begins with the implementation of baseline models using the AISY framework [16], which provides a standardized environment for conducting profiling attacks and evaluating performance using metrics such as Guessing Entropy (GE) and Success Rate (SR) [1], [6]. Initial experiments employ multilayer perceptron (MLP) architectures to establish a reference performance level. These models, while effective to some extent, are limited in their ability to capture localized temporal features in power traces [2], [5], [12].To address these limitations, convolutional neural networks (CNNs) are introduced as the primary modeling approach.

CNNs are well-suited for side-channel analysis due to their ability to automatically extract hierarchical features from one-dimensional signals such as power traces. Through convolutional filtering and pooling operations, CNNs capture local dependencies and patterns associated with cryptographic operations, while also providing robustness to noise and temporal misalignment [2], [5], [8], [12], [15]. The transition from MLP to CNN architectures forms a key aspect of the proposed approach, enabling improved leakage extraction and classification performance.

In addition to framework-based experimentation, a standalone implementation is developed to directly process the ASCAD dataset.The standalone pipeline includes trace normalization, reshaping for convolutional input, and transformation of labels into leakage classes based on the HW model [2], [5], [8]. This dual approach of leveraging both AISY and custom implementations thus ensures both reproducibility and extensibility of the experimental setup.

Trained Models are evaluated using both classification accuracy and rank-based metrics. While accuracy measures the model's ability to predict leakage classes, rank-based metrics provide a more meaningful assessment in the context of SCA by evaluating the position of the correct key hypothesis among all candidates. This aligns with standard evaluation practices in profiling attacks, where the ultimate objective is to minimize the rank of the correct key and achieve successful key recovery

B. Dataset Description of ASCAD

The ASCAD dataset consists of power consumption traces acquired from a hardware implementation of the Advanced Encryption Standard (AES), specifically targeting leakage during the SubBytes operation. Each trace represents the temporal evolution of power consumption during a single encryption execution and is accompanied by metadata including plaintext inputs, cryptographic keys, and precomputed labels corresponding to intermediate values [16]. It contains power traces acquired from real hardware implementation of AES along with corresponding metadata like plaintexts and labels which represent intermediate values and can be used to introduce noise while training the models. Each trace captures the temporary change in power consumption during encryption and acts as the input learning model.

Given the presence of noise and variability in measurements preprocessing steps are used to normalize traces and prepare them for Machine learning Analysis. Since ASCAD is designed for profiling SCA's, they are typically partitioned into two distinct subsets: a profiling set and an attack set. The profiling traces are used to train machine learning models by learning the relationship between power consumption and secret-dependent intermediate values, while the attack traces are used to evaluate the trained model's ability to generalize and recover key-related information.

The dataset provides labels associated with intermediate computations of the AES algorithm. These labels are often transformed using a leakage model such as the Hamming Weight (HW), resulting in classification targets with reduced dimensionality (e.g., 9 classes corresponding to HW values from 0 to 8).

ASCAD includes different variants, such as fixed-key and variable-key configurations. In the fixed-key setting, the encryption key remains constant across all traces, simplifying the learning problem, whereas in the variable-key setting, the key changes across traces, increasing the difficulty of the attack and requiring the model to generalize more effectively.

C. Leakage Modelling, Data Preprocessing, Normalization and Reshaping

Leakage modeling gives the mathematical relationship between secret-dependent intermediate values and the observable side-channel signal. In power-based SCA, the leakage is assumed to arise from data-dependent switching activity in CMOS circuits shown in instantaneous change in power usage. A general leakage model is defined as:

$$L(t) = f(v(t)) + n(t)$$

$L(t)$ = Observed Leakage ; $f(v(t))$: leakage functio of internal intermediate value ; $n(t)$ is the noise

For AES, intermediate value is:

$$v = Sbox(P \oplus K)$$

where P: Plaintext byte; K: 'Secret' key byte ; Sbox: AES Substitution operation

The Hamming Weight model:

$$HW(v) = \sum_{i=0}^{7} b_i$$

where b_i are bits of (v), thus converting static power into bits.

The Hamming Distance Model:

$$HD = HD(v_1, v_2) = \sum_{i=0}^{7} \left(b_i^{(1)} \oplus b_i^{(2)} \right)$$

which models dynamic power consumption based on bit transitions between successive states.

The classification target is thus:

$$y = HW(Sbox(P \oplus K)) \in \{0, 1, \ldots, 8\}$$

Hence $X \to Y$, i.e from a power problem to a classification problem.

Raw Side channel traces are noise due to measurement conditions, device characteristics, and execution jitter. Preprocessing is needed to enhance the signal-to-noise ratio (SNR) and enable stable learning. The dataset can be exhibited by:

$$X \in \mathbb{R}^{N \times T}$$

Where N: No of traces and T: Time sample per trace. Thus we achieve Noise reduction, Feature alignment and Scale normalization.

Additionally, the label vector:

$$Y \in \{0, \ldots, 255\}$$

is transformed using the leakage model into:

$$Y' \in \{0, \ldots, 8\}$$

Normalization is done to standardize data distribution of power traces for stability during training without which large magnitudes dominate learning. Used method was Z-score normalisation:

$$X_{\text{norm}}(i, j) = \frac{X(i, j) - \mu_j}{\sigma_j}$$

Where $\mu_j \to$ mean of time sample and $\sigma_j \to$ standard deviation.

Reshaping is required to adapt the one-dimensional power traces into a format compatible with convolutional neural networks (CNNs). While raw traces are represented as:

$$X \in \mathbb{R}^{N \times T}$$

CNNs expect input tensors with an explicit channel dimension:

$$X \in \mathbb{R}^{N \times T \times C}$$

; C is number of channels ($C = 1$ for power traces)

$$X_{\text{reshaped}} = \text{reshape}(X, (N, T, 1))$$

Applying Convolutional layers to apply filters:

$$y[n] = \sum_{k=0}^{K-1} w[k] \cdot x[n - k]$$

where $w[k]$ is the convolution kernel of size K.

D. Training Procedure

Key terms:

Trace: Each input sample X_i corresponds to a single power trace representing one AES execution.

Epoch: An epoch is defined as one complete pass over the entire training dataset D.

Iteration: One iteration corresponds to processing a single mini-batch and updating model parameters.

Batch Size Effect: Gradient estimate:

$$\nabla L \approx \frac{1}{B} \sum_{i \in \text{batch}} \nabla l_i$$

Train-validation split: A portion of profiling traces is used for validation during training.

The NN models are trained on preprocessed power traces and corresponding leakage labels derived from the Hamming Weight (HW) model. Both framework-based (AISY) and standalone implementations employ similar optimization principles, differing primarily in architectural flexibility and execution control [1], [2], [5], [8], [12].

Each training sample corresponds to a single power trace, which represents the power consumption of one AES encryption. The dataset is processed in mini-batches of size B, where each batch is used to compute an estimate of the gradient for parameter updates. One epoch is defined as a complete pass through the entire training dataset, and training is performed over multiple epochs to ensure convergence. The total number of parameter update steps is given by:

$$\left(\frac{N}{B} \right) \times \text{epochs}$$

Model Training Objective

The goal of training is to learn a parametric function:

$$f_\theta : \mathbb{R}^T \to \mathbb{R}^C$$

where θ denotes model parameters and $C = 9$ is the number of classes. The output is a probability distribution over leakage classes:

$$\hat{y}_i = \text{softmax}(z_i)$$

where:

$$\text{softmax}(z_j) = \frac{\exp(z_j)}{\sum_{k=1}^{C} \exp(z_k)}$$

Loss Function

The models are trained using the categorical cross-entropy loss:

$$L(\theta) = -\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

where $y_{i,c}$ is the one-hot encoded ground truth label and $\hat{y}_{i,c}$ is the predicted probability for class c [2], [5], [12].

For integer labels, the sparse version is used:

$$L(\theta) = -\sum_{i=1}^{N}\log(\hat{y}_{i,y_i})$$

Batch Training

Training is performed in mini-batches of size B, where each iteration processes a subset:

$$B \subset D, \quad |B| = \text{batch size}$$

This reduces memory usage and stabilizes gradient estimation. The dataset is iterated over multiple epochs, where one epoch corresponds to a complete pass through the training set.

AISY-Based Training

Within the AISY framework, predefined scripts automate the training process, including dataset loading, batching, model instantiation, and evaluation. MLP-based models are trained using fully connected layers, where each layer performs:

$$h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right)$$

where σ is a non-linear activation function (e.g., ReLU) [1], [6].

These baseline models provide initial performance metrics such as Guessing Entropy (GE) and Success Rate (SR) [16].

CNN-Based Training (Proposed Approach)

In the custom implementation, CNN models are trained to exploit the temporal structure of power traces. The convolution operation is defined as:

$$y[n] = \sum_{k=0}^{K-1} w[k]\cdot x[n-k]$$

where $w[k]$ is the kernel and K is the kernel size.

Multiple convolutional layers learn hierarchical representations:

Feature Maps $\rightarrow$ Flatten $\rightarrow$ Dense Layers $\rightarrow$ Softmax Output

CNN training follows the same optimization procedure but benefits from:

- localized feature extraction
- reduced parameter sharing
- robustness to noise and misalignment

which significantly improves learning efficiency for side-channel data [2], [5], [8], [12], [15].

During training, Accuracy is noted using:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{y}_i = y_i)$$

however it proves to be insufficient and are thur evaluated using rank metrics

V. Pseudocode

A combination of programs from AISY were run on ASCAD databases to develop an understanding of data framework, objectives and goals required for our program. The subsequently developed programs by us are as followed:

Algorithm 1: Standalone CNN-Based Side-Channel Attack [Script_aes_cnn.py]

Input: ASCAD dataset (D)
Output: Trained model, prediction accuracy, average rank

Load ASCAD dataset;
Extract profiling traces (X_p), labels (Y_p);
Extract attack traces (X_a), labels (Y_a);
Convert labels using Hamming Weight:;
foreach $y \in Y_p, Y_a$ **do**
    $y \leftarrow HW(y)$;
Normalize traces:;
Compute mean (μ) and std (σ) from (X_p);
$X_p \leftarrow (X_p - \mu)/\sigma$;
$X_a \leftarrow (X_a - \mu)/\sigma$;
Reshape for CNN:;
$X_p \leftarrow \text{reshape}(X_p, (N_p, T, 1))$;
$X_a \leftarrow \text{reshape}(X_a, (N_a, T, 1))$;
Initialize CNN model:;
Conv1D $\rightarrow$ Pool $\rightarrow$ Conv1D $\rightarrow$ Pool $\rightarrow$ Flatten;
Dense $\rightarrow$ Softmax (9 classes);
Compile model using Adam optimizer and cross-entropy loss;
Train model:;
for $epoch = 1$ **to** E **do**
    Train on mini-batches;
    Validate using validation split;
Prediction phase:;
$P \leftarrow \text{model.predict}(X_a)$;
Compute rank:;
foreach P_i **do**
    rank $\leftarrow$ number of classes with probability $>$ true class;
Compute average rank;
Return accuracy and average rank;

The proposed approach implements a standalone CNN-based side-channel analysis pipeline, where raw ASCAD power traces are manually preprocessed and used to train a convolutional neural network. Unlike framework-based methods, this approach provides full control over data preprocessing, leakage modeling, and training configuration. The model

is trained to classify Hamming Weight leakage classes, and its effectiveness is evaluated using rank-based metrics.

Algorithm 2: AISY-Based CNN Side-Channel Attack [Script_aes_cnn_new.py]

Input: ASCAD dataset, AISY framework
Output: Guessing Entropy (GE), Success Rate (SR)

Initialize AISY framework;
Set directories:;
resources folder;
database folder;
dataset folder;
Configure dataset:;
Select ASCAD dataset;
Set number of profiling traces (N_p);
Set number of attack traces (N_a);
Define leakage model:;
Use Hamming Weight (HW);
Select target AES byte;
Set training parameters:;
Batch size (B);
Number of epochs (E);
Build CNN model:;
Conv1D $\rightarrow$ Pool $\rightarrow$ Conv1D $\rightarrow$ Pool;
Flatten $\rightarrow$ Dense $\rightarrow$ Softmax;
Compile model using Adam optimizer;
Assign CNN model to AISY framework;
Run attack:;
AISY automatically performs:;
Load dataset;
Preprocess traces;
Train model;
Predict on attack traces;
Evaluate attack:;
Compute Guessing Entropy (GE);
Compute Success Rate (SR);
Return GE and SR;

To establish a baseline, the AISY framework is utilized to perform profiling side-channel attacks using predefined pipelines. The framework automates dataset handling, model training, and evaluation using metrics such as Guessing Entropy and Success Rate. This allows comparison between standard MLP-based approaches and the proposed CNN-based method.

VI. Results

A. MLP-Based Baseline (AISY Framework)

The script _aes_neural_network_parameters.py employs a multilayer perceptron (MLP) with approximately 91k parameters. Training was conducted on 200,000 profiling traces with validation on 10,000 traces for 15 epochs. The training accuracy gradually improved from 26.55% to 28.89%, while validation accuracy remained relatively stable around 27–28%.

Despite this improvement, the attack performance remained limited, with:

$$GE = 95.93, \quad SR = 0.02$$

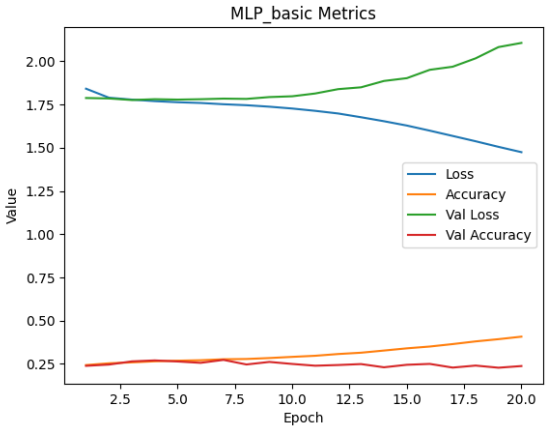
indicating that the correct key was rarely ranked highly.

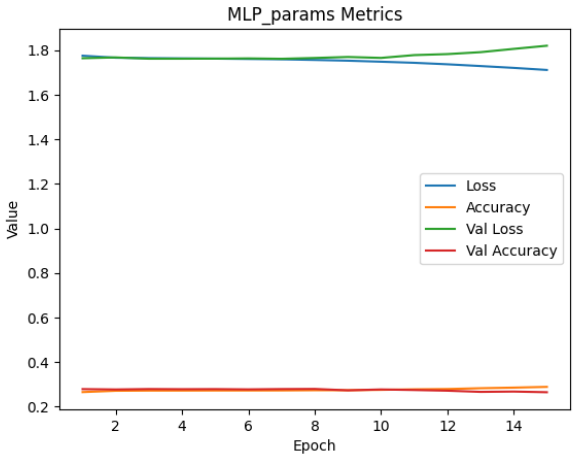
The `script_aes.py` experiment utilized a deeper MLP architecture with approximately 262k parameters. Over 20 epochs, training accuracy increased significantly from 24.36% to 40.78%, demonstrating improved fitting capability. However, validation accuracy remained low (≈23–27%), suggesting overfitting.

This is further reflected in poor attack performance:

$$GE = 123.76, \quad SR = 0.0$$

indicating ineffective key recovery despite higher training accuracy.



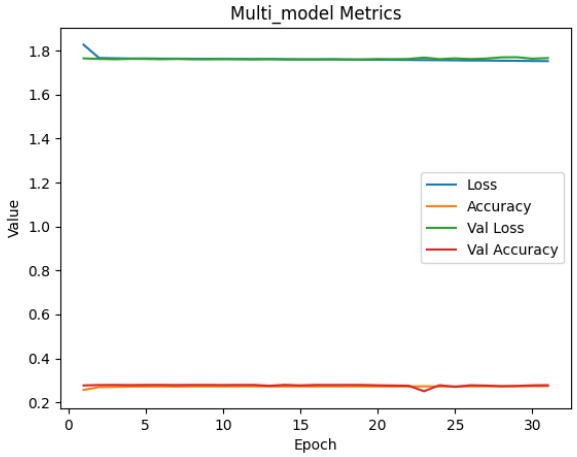


B. Multi-Model Evaluation

The `script_aes_multi_model.py` script evaluated multiple architectures, including both MLP and CNN variants. The CNN-based configurations in this setup were relatively shallow, with parameter counts ranging from approximately 3k to 725k.

Training over 50 epochs showed minimal improvement in validation accuracy, which remained around 27–28% throughout training. The loss values also plateaued early, indicating limited learning of discriminative leakage features.

Overall, these results demonstrate that shallow CNN architectures within the AISY framework do not significantly outperform MLP baselines under the given configuration.



C. Standalone CNN Model (script_aes_cnn.py)

A custom CNN model with approximately 70k parameters was implemented to improve feature extraction. The model consists of convolutional and pooling layers followed by dense classification layers.

Training was performed for 5 epochs on 200,000 traces. The model achieved:

- Training accuracy $\approx 27.5\%$
- Validation accuracy $\approx 28.07\%$

giving:

$$GE = 151.02, \quad SR = 0.0$$

This indicates that although the CNN architecture slightly stabilized classification accuracy, it failed to extract sufficiently strong leakage information for key recovery.

D. Improved CNN Model (script_aes_cnn_new.py)

A more advanced CNN model was developed with approximately 1.1 million parameters and trained on a reduced dataset (18,000 profiling traces and 2,000 validation traces). Training over 10 epochs showed significant improvement in training accuracy, reaching:

$$\text{Final Training Accuracy} = 72.33\%$$

However, validation accuracy degraded over epochs (from $\sim$27% to $\sim$21%), indicating overfitting.

Despite this, attack performance improved substantially in terms of rank:

$$\text{Average Rank} = 2.2764$$

This represents a significant improvement compared to earlier models, indicating that the correct key is typically among the top candidates.

The rank progression plot shows the variation of rank across the first 500 attack traces. The rank frequently attains low values (0–3), with occasional spikes to higher values.

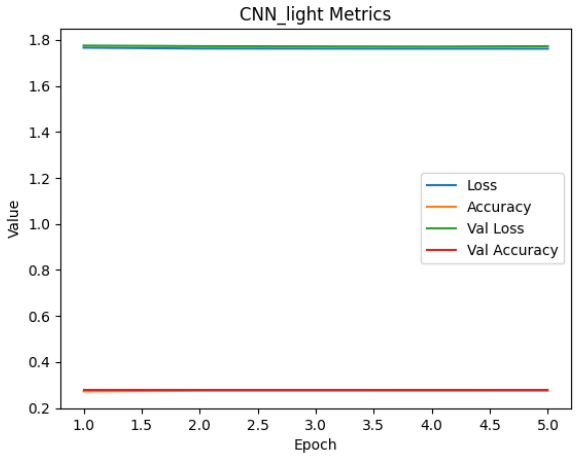
This behavior indicates:

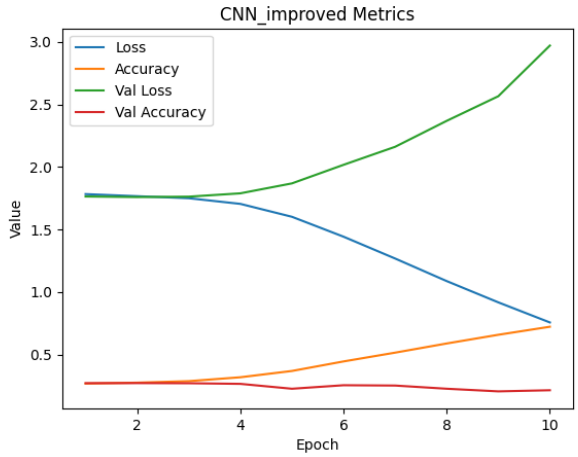
- partial but inconsistent key recovery
- sensitivity to individual trace variations
- presence of noise and model uncertainty

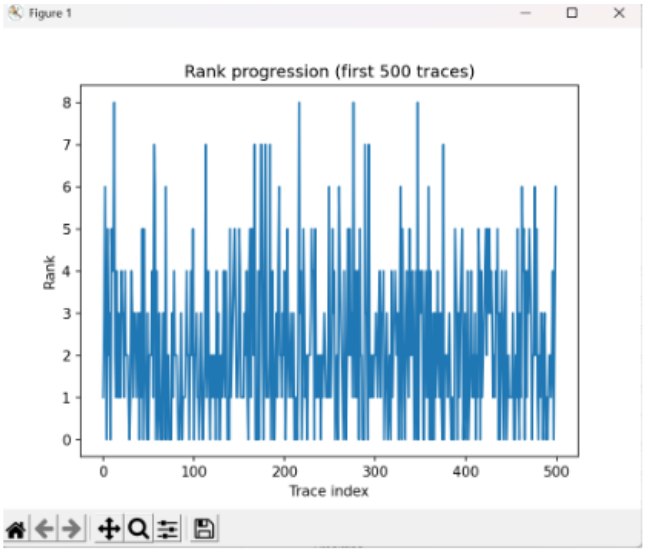
The rank progression plot illustrates the rank's variability across the initial 500 attack traces. The rank often registers low values (typically 0–3) but sporadically spikes to higher figures.

This pattern suggests:

- Inconsistent, partial key recovery.
- High sensitivity to variations in individual traces.
- The influence of noise and inherent model uncertainty.







VII. Key Observations and Discussion

Key Observations

MLP models achieve moderate classification accuracy but fail in key recovery, as indicated by high GE and low SR.

Higher training accuracy does not guarantee better attack performance, highlighting the importance of rank-based metrics.

Initial CNN models show limited improvement, suggesting insufficient architectural depth or feature extraction capability.

The improved CNN model significantly reduces rank, demonstrating enhanced ability to capture leakage patterns.

Overfitting remains a major challenge, particularly in the improved CNN model trained on a smaller dataset.

A summarised table:

Model	Architecture	Epochs	Final Train Acc	Final Val Acc	GE	SR	Avg Rank	Key Observation
MLP_params	MLP (~91k)	15	0.2889	0.2649	95.93	0.02	High	Moderate learning, weak key recovery
MLP_basic	MLP (~262k)	20	0.4078	0.2379	123.76	0	Very High	Overfitting, poor generalization
Multi_model	Mixed (MLP+CNN)	50	~0.273	~0.278	—	—	High	Plateau, no meaningful learning
CNN_light	CNN (~70k)	5	0.275	0.2807	151.02	0	Very High	Underfitting, ineffective model
CNN_improved	CNN (~1.1M)	10	0.7233	0.216	—	—	**2.27**	Strong learning, best key ranking

Thus training limitations due to device limitations Affected the end result, however the improved CNN script provided satisfactory results

VIII. Discussion

This work investigated the effectiveness of deep learning–based side-channel analysis (SCA) on AES using the ASCAD dataset, with a focus on comparing baseline multilayer perceptron (MLP) models within the AISY framework against custom convolutional neural network (CNN) architectures. Experimental results show that MLP based approaches achieve moderate classification accuracy but fail to translate into effective key recovery in the absence of very capable hardware, however improvements upto 40%. The high GE and low SR highlighted a key limitation of fully connected architectures in capturing localised temporal dependencies present in power traces.

The introduction of CNN-based models provided improved feature extraction capabilities. Initial lightweight CNN implementations showed only marginal gains, suggesting that shallow architectures are insufficient for capturing complex leakage patterns. However, the enhanced CNN model demonstrated a significant reduction in key rank (average rank $\approx$ 2.27), indicating that the correct key is frequently among the top candidates . This confirms the suitability of convolutional architectures for modeling side-channel leakage.

It is important to note that classification accuracy alone is not a reliable indicator of attack success. Models with relatively low accuracy were still able to achieve improved key ranking, emphasizing the importance of rank-based evaluation metrics such as GE and SR in SCA.

Several Key challenges were observed. The improved CNN model exhibited overfitting while training accuracy increased when validation accuracy degraded. This shows limited generalization capabilities and shows that model complexity, dataset size, and regularization strategies must be carefully balanced.

References

[1] B. Das et al., "Machine Learning-Based Detection of Microarchitectural Side-Channel Attacks," Available: https://www.cse.iitb.ac.in/~biswa/AIMLSYS.pdf

[2] "Deep Learning for Side-Channel Analysis," Available: https://arxiv.org/pdf/2404.11284

[3] "Side-Channel Attacks in Multi-Tenant Systems," Available: https://www.proofs-workshop.org/2021/papers/paper9.pdf

[4] "Cache Attacks and Countermeasures: The Case of AES," IACR Cryptology ePrint Archive, Available: https://eprint.iacr.org/2018/053.pdf

[5] "Recent Advances in Side-Channel Analysis," Available: https://arxiv.org/pdf/2603.10726

[6] "Microarchitectural Attack Detection Using ML," Available: https://arxiv.org/pdf/2603.18647

[7] "Cache-Based Side Channel Attacks: Prime+Probe and Flush+Reload," Available: https://pure.port.ac.uk/ws/files/7574489/UKCI_2017_PAPER.pdf

[8] "Deep Learning-Based Side Channel Analysis Survey," Available: https://arxiv.org/pdf/2103.14244

[9] "IACR ePrint Report 2025/014," Available: https://eprint.iacr.org/2025/014.pdf

[10] "Side-Channel Analysis Using Neural Networks," Available: https://arxiv.org/pdf/2312.11094

[11] "Machine Learning for Hardware Security," Available: https://arxiv.org/pdf/2309.13170

[12] C. Wang et al., "Hardware-Based Side-Channel Detection," Available: https://homepages.uc.edu/~wang2ba/files/pub/host23_chenggang.pdf

[13] "ASCAD: A Dataset for Side-Channel Analysis," Available: https://arxiv.org/pdf/1605.00681

[14] "Deep Learning for Physical Side-Channel Attacks," Available: https://arxiv.org/pdf/1611.03748

[15] "Survey on Side-Channel Attacks and Countermeasures," Available: https://www.sciencedirect.com/science/article/abs/pii/S0026269223002148

[16] “AISY Framework,” Available: https://github.com/aisylab/aisy-framework
[17] “ASCAD Dataset,” Available: https://github.com/ANSSI-FR/ASCAD